Aula 10 — Analisadores Sintáticos Preditivos

Compiladores I

FACOM - Faculdade de Computação

Bloco 4: Análise Sintática

Duração estimada: 70–80 minutos

Revisão da Aula 9

- Análise sintática: tokens $\rightarrow$ árvore sintática;
- Estratégias descendente e ascendente.

Pergunta rápida

Qual das duas estratégias constrói a árvore "de cima para baixo"?

Objetivos de aprendizagem

- Calcular os conjuntos FIRST e FOLLOW de uma gramática;
- Construir uma tabela de análise preditiva LL(1);
- Transformar uma gramática para torná-la adequada à análise preditiva.

Sumário da aula

Conjuntos FIRST e FOLLOW

Análise preditiva e tabelas LL(1)

Conjunto FIRST

Definição

$FIRST(\alpha)$: conjunto de todos os terminais que podem iniciar alguma cadeia derivada de α (inclui ε se α pode gerar a cadeia vazia).

Exemplo —

Para `fator -> NUM | ID | '(' expr ')'`:
$FIRST(\text{fator}) = \{\text{NUM}, \text{ID}, (\}$.

Pergunta rápida

Se `A -> B c` e `B` pode gerar a cadeia vazia (ε), o que deve entrar em $FIRST(A)$ além de $FIRST(B)$?

Conjunto FOLLOW

Definição

FOLLOW(A): conjunto de terminais que podem aparecer imediatamente **depois** de A em alguma derivação a partir do símbolo inicial.

Exemplo —

Para `expr -> expr '+' termo | termo` e `expr` sendo usado como `'(' expr ')'`: o símbolo `')'` pertence a *FOLLOW*(expr).

FOLLOW é essencial para decidir quando aplicar uma produção que gera ε.

Sumário da aula

Conjuntos FIRST e FOLLOW

Análise preditiva e tabelas LL(1)

Construindo a tabela de análise preditiva

Regra de preenchimento

Para cada produção $A \to \alpha$: para cada terminal $a \in FIRST(\alpha)$, adicione $A \to \alpha$ na célula $M[A, a]$. Se $\varepsilon \in FIRST(\alpha)$, adicione $A \to \alpha$ para cada $b \in FOLLOW(A)$.

A tabela $M[\text{não-terminal}, \text{terminal}]$ orienta o parser: "estando em A e vendo o token a, qual produção aplicar?"

Gramáticas LL(1)

Definição

Uma gramática é LL(1) se, para cada não-terminal, cada célula da tabela de análise preditiva tem **no máximo uma** produção — ou seja, o parser sempre sabe qual produção escolher olhando apenas 1 token à frente.

- "LL": lê a entrada da esquerda (*Left*) para a direita, produz derivação mais à esquerda (*Leftmost*);
- "(1)": usa 1 símbolo de lookahead.

Gramáticas que não são LL(1)

- Gramáticas ambíguas nunca são LL(1);
- Gramáticas com recursão à esquerda direta ou indireta não são LL(1);
- Gramáticas onde duas produções alternativas começam com o mesmo prefixo também causam conflito (resolvido por fatoração à esquerda).

Pergunta rápida

A gramática `cmd -> 'if' expr 'then' cmd | 'if' expr 'then' cmd 'else' cmd` tem um problema para ser LL(1). Qual?

Recuperação de erros na análise preditiva

- Se $M[A, a]$ está vazia, há um erro sintático — o token a não era esperado naquele ponto;
- Estratégia simples: reportar o erro e "sincronizar", descartando tokens até encontrar um símbolo em $FOLLOW(A)$, permitindo continuar a análise;
- Boas mensagens de erro citam a linha/coluna e o token esperado x encontrado.

Desafio em sala

Desafio em sala

Para a gramática `S -> 'a' A | 'b'` e `A -> 'c' A |` ε: calcule $FIRST(S)$, $FIRST(A)$ e $FOLLOW(A)$, e monte a tabela de análise preditiva M.

Síntese da aula

- *FIRST* e *FOLLOW* são a base para construir a tabela de análise preditiva;
- Uma gramática LL(1) permite decidir a produção correta com 1 token de lookahead;
- Recursão à esquerda e prefixos comuns impedem que uma gramática seja LL(1).

Próxima aula: implementação da análise descendente recursiva.

Teste rápido — Aula 10

Questões teóricas

1. O que significa a sigla "LL(1)"?
2. Por que uma gramática ambígua nunca pode ser LL(1)?

Questão prática

3. Para `fator -> NUM | ID | '(' expr ')'`, calcule *FIRST*(fator).